



PTX information

from : <https://docs.nvidia.com/cuda/parallel-thread-execution/#>

a PTX program specifies the execution of a given thread of a parallel thread array. A *cooperative thread array*, or CTA, is an array of threads that execute a kernel concurrently or in parallel.

Threads within a CTA can communicate with each other. To coordinate the communication of the threads within the CTA, one can specify synchronization points where threads wait until all threads in the CTA have arrived.

Typically, a warp has 32 threads. Some applications may be able to maximize performance with knowledge of the warp size, so PTX includes a run-time immediate constant, `WARP_SZ`, which may be used in any instruction where an immediate operand is allowed.

A multiprocessor consists of multiple *Scalar Processor (SP)* cores, a multithreaded instruction unit, and on-chip shared memory (for report).

Syntax

1. Source Format :

PTX is case sensitive and uses lowercase for keywords.

Each PTX module must begin with a `.version` directive specifying the PTX language version, followed by a `.target` directive specifying the target architecture assumed

All whitespace characters are equivalent; whitespace is ignored except for its use in separating tokens in the language.

Comments in PTX are treated as whitespace. `//` or `/* */`

2. Statements :

A PTX statement is either a directive or an instruction. Statements begin with an optional label and end with a semicolon.

2.1. Directive Statements :

Directive keywords begin with a dot, so no conflict is possible with user-defined identifiers.

<code>.address_size</code>	<code>.explicitcluster</code>	<code>.maxnreg</code>	<code>.section</code>
<code>.alias</code>	<code>.extern</code>	<code>.maxntid</code>	<code>.shared</code> : Addressable memory, accessible to all

			threads in the cluster throughout the lifetime of the CTA that defines it.
<code>.align</code>	<code>.file</code>	<code>.minnctapersm</code>	<code>.sreg</code> : Special registers. Read-only; pre-defined; platform-specific.
<code>.branchtargets</code>	<code>.func</code>	<code>.noreturn</code>	<code>.target</code>
<code>.callprototype</code>	<code>.global</code> : Global memory, shared by all threads.	<code>.param</code> : Kernel parameters, defined per-grid; or Function or local parameters, defined per-thread.	<code>.tex</code> : Global texture memory (deprecated)
<code>.calltargets</code>	<code>.loc</code>	<code>.pragma</code>	<code>.version</code>
<code>.common</code>	<code>.local</code> : Local memory, private to each thread It is typically standard memory with cache.	<code>.reg</code> : Registers, fast.	<code>.visible</code>
<code>.const</code> : Shared, read-only memory.	<code>.maxclusterrank</code>	<code>.reqnctapercluster</code>	<code>.weak</code>
<code>.entry</code>	<code>.maxnctapersm</code>	<code>.reqntid</code>	

2.2. Instruction Statements :

Instruction keywords are listed in the under table. All instruction keywords are reserved tokens in PTX

Instructions are formed from an instruction opcode followed by a comma-separated list of zero or more operands, and terminated with a semicolon. Operands may be register variables, constant expressions, address expressions, or label names.

The destination operand is first, followed by source operands.

```
add.s64    %rd22, %rd11, %rd12;
mad.lo.s32 %r30, %r17, %r2, %r36;
```

<code>abs</code>	<code>cvta</code>	<code>membar</code>	<code>setp</code>	<code>vabsdiff</code>
<code>activemask</code>	<code>discard</code>	<code>min</code>	<code>shf</code>	<code>vabsdiff2</code>
<code>add</code>	<code>div</code>	<code>mma</code>	<code>shfl</code>	<code>vabsdiff4</code>
<code>addc</code>	<code>dp2a</code>	<code>mov</code>	<code>shl</code>	<code>vadd</code>
<code>alloca</code>	<code>dp4a</code>	<code>movmatrix</code>	<code>shr</code>	<code>vadd2</code>
<code>and</code>	<code>elect</code>	<code>mul</code>	<code>sin</code>	<code>vadd4</code>
<code>applypriority</code>	<code>ex2</code>	<code>mul24</code>	<code>slct</code>	<code>vavrg2</code>
<code>atom</code>	<code>exit</code>	<code>multimem</code>	<code>sqr</code>	<code>vavrg4</code>

bar	fence	nanosleep	st	vmad
barrier	fma	neg	stackrestore	vmax
bfe	fns	not	stacksave	vmax2
bfi	getctarank	or	stmatrix	vmax4
bfind	griddepcntrol	pmevent	sub	vmin
bmsk	isspacep	popc	subc	vmin2
bra	istypep	prefetch	suld	vmin4
brev	ld	prefetchu	suq	vote
brkpt	ldmatrix	prmt	sured	vset
brx	ldu	rcp	sust	vset2
call	lg2	red	szext	vset4
clz	lop3	redux	tanh	vshl
cnot	mad	rem	tcgen05	vshr
copysign	mad24	ret	tensormap	vsub
cos	madc	rsqrt	testp	vsub2
clusterlaunchcontrol	mapa	sad	tex	vsub4
cp	match	selp	tld4	wgmma
createpolicy	max	set	trap	wmma
cvt : takes a variety of operand types and sizes, as its job is to convert from nearly any data type to any other data type (and size)	mbarrier	setmaxnreg	txq	xor

Instructions have an optional guard predicate which controls conditional execution. The guard predicate follows the optional label and precedes the opcode, and is written as `@p`, where `p` is a predicate register. The guard predicate may be optionally negated, written as `!p`.

```
@%p4 bra $L__BB0_4;
```

- `%p4` is a **predicate register** that holds a Boolean value (true/false).
- The `@` in front tells PTX: *only execute the instruction if this predicate is true.*
- If it were `!%p4`, it would mean *execute if the predicate is false.*

bra → This is the **opcode**, short for "branch." It tells the program to jump to a label.

3. Identifiers :

User-defined identifiers follow extended C++ rules: they either start with a letter followed by zero or more letters, digits, underscore, or dollar characters; or they start with an underscore, dollar, or percentage character followed by one or more letters, digits, underscore, or dollar characters:

```
followsym:  [a-zA-Z0-9_$]
identifier: [a-zA-Z]{followsym}* | {[_$]{followsym}+
```

PTX does not specify a maximum length for identifiers and suggests that all implementations support a minimum length of at least 1024 characters.

<code>%aggr_smem_size</code>	<code>%dynamic_smem_size</code>	<code>%lanemask_gt</code>	<code>%reserved_smem_offset_begin</code>
<code>%clock</code>	<code>%envreg<32></code>	<code>%lanemask_le</code>	<code>%reserved_smem_offset_cap</code>
<code>%clock64</code>	<code>%globaltimer</code>	<code>%lanemask_lt</code>	<code>%reserved_smem_offset_end</code>
<code>%cluster_ctaid</code>	<code>%globaltimer_hi</code>	<code>%nclusterid</code>	<code>%smid</code>
<code>%cluster_ctarank</code>	<code>%globaltimer_lo</code>	<code>%nctaid</code>	<code>%tid</code>
<code>%cluster_nctaid</code>	<code>%gridid</code>	<code>%nsmid</code>	<code>%total_smem_size</code>
<code>%cluster_nctarank</code>	<code>%is_explicit_cluster</code>	<code>%ntid</code>	<code>%warpid</code>
<code>%clusterid</code>	<code>%laneid</code>	<code>%nwarpid</code>	<code>WARP_SZ</code>
<code>%ctaid</code>	<code>%lanemask_eq</code>	<code>%pm0, ..., %pm7</code>	
<code>%current_graph_exec</code>	<code>%lanemask_ge</code>	<code>%reserved_smem_offset_<2></code>	

Integer constants are 64-bits in size and are either signed or unsigned, i.e., every integer constant has type `.s64` or `.u64`. The signed/unsigned nature of an integer constant is needed to correctly evaluate constant expressions containing operations such as division and ordered comparisons, where the behavior of the operation depends on the operand types

Integer literals may be written in decimal, hexadecimal, octal, or binary notation. The syntax follows that of C. Integer literals may be followed immediately by the letter `u` to indicate that the literal is unsigned.

```
hexadecimal literal: 0[xX]{hexdigit}+U?
octal literal:       0{octal digit}+U?
binary literal:      0[bB]{bit}+U?
decimal literal      {nonzero-digit}{digit}*U?
```

for floating point constants :

- In C you can write `3.14f` for single precision, or `3.14` for double.
- In PTX, **there's no suffix**: all numeric literals are treated as **64-bit double**.

Sometimes you need the **exact IEEE 754 bit pattern**, because decimal constants can't represent every floating-point number exactly. PTX allows this with **hex constants**.

- **Double-precision (64-bit)**: starts with `0d` or `0D` + **16 hex digits**
- **Single-precision (32-bit)**: starts with `0f` or `0F` + **8 hex digits**
- **Predicate-type operands**: some instructions take predicates, e.g., `@p` before `bra`, `@p` for `add` with conditional execution.
- **Predicate initializers**: when you assign an initial value to a predicate register:

```
mov.pred %p1, 1;    // sets %p1 to True or any non zero integer
mov.pred %p2, 0;    // sets %p2 to False
```

This is why you sometimes see PTX instructions like:

```
@l bra $L__BB0_4;
```

It's just a shortcut for "always branch" without needing a predicate register.

- A **constant expression** is something that the assembler can fully evaluate **without running the program**.
- Examples: numeric literals (`42` , `3.14`), or formulas like `4 + 5` , `1 << 3` , `(2*3 + 1)` .
- These are **not instructions**. They are **values** you can assign to registers, memory, or predicate registers at assembly time.

```
// assign constant expressions to integer registers
mov.u32 %r1, 3 + 4 * 2;          // evaluated as 3 + (4*2) = 11
mov.u32 %r2, (1 << 5) | 3;      // shift and bitwise OR, result = 35
mov.pred %p1, (4 > 2) ? 1 : 0;  // ternary, sets %p1 = 1 (true)
```

- Literals are signed unless unsigned is needed to prevent overflow, or unless the literal uses a `U` suffix. For example:
 - `42` , `0x1234` , `0123` are signed.
 - `0xfabc123400000000` , `42U` , `0x1234U` are unsigned.
- Unary plus and minus preserve the type of the input operand. For example:
 - `+123` , `1` , `(-42)` are signed.
 - `1U` , `0xfabc123400000000` are unsigned.

4. State Spaces, Types, and Variables :

A state space is a storage area with particular characteristics. All variables reside in some state space. The characteristics of a state space include its size, addressability, access speed, access rights, and level of sharing between threads.

```
// Kernel with input parameter
.visible .entry add_kernel(
```

```

    .param .u32 input_val
)
{
    .reg .u32 r1;

    // Load the kernel input
    ld.param.u32 r1, [input_val]; // only way to access input_val

    // Increment and store in local register
    add.u32 r1, r1, 1;
}

// Device function with parameter
.func add_func(
    .param .u32 x
)
{
    .reg .u32 r1;

    // Load parameter
    ld.param.u32 r1, [x];

    // Increment
    add.u32 r1, r1, 1;

    // Store back
    st.param.u32 [x], r1;
}

.reg .u64 x_addr;
mov.u64 x_addr, x; // x now has an address in .local memory

```

When the limit is exceeded, register variables will be spilled to memory, causing changes in performance. For each architecture, there is a recommended maximum number of registers to use (for example for my target architecture : jetson orin nano) Registers Per Block: 65536 Registers Per Multiprocessor: 65536

Each kernel function definition includes an optional list of parameters. These parameters are addressable, read-only variables declared in the `.param` state space. Values passed from the host to the kernel are accessed through these parameter variables using `ld.param` instructions. The kernel parameter variables are shared across all CTAs from all clusters within a grid.

The address of a kernel parameter may be moved into a register using the `mov` instruction. The resulting address is in the `.param` state space and is accessed using `ld.param` instructions.

Example

```
.entry foo ( .param .b32 N, .param .align 8 .b8 buffer[64] )
{
    .reg .u32 %n;
    .reg .f64 %d;

    ld.param.u32 %n, [N];
    ld.param.f64 %d, [buffer];
    ...
}
```

Example

```
.entry bar ( .param .b32 len )
{
    .reg .u32 %ptr, %n;

    mov.u32 %ptr, len;
    ld.param.u32 %n, [%ptr];
    ...
}
```

Kernel function parameters may represent normal data values, or they may hold addresses to objects in constant, global, local, or shared state spaces. In the case of pointers, the compiler and runtime system need information about which parameters are pointers, and to which state space they point. Kernel parameter attribute directives are used to provide this information at the PTX level.

```
.param .type .ptr .space .align N varname
.param .type .ptr .align N varname

.space = { .const, .global, .local, .shared };
.entry foo ( .param .u32 param1,
             .param .u32 .ptr.global.align 16 param2,
             .param .u32 .ptr.const.align 8 param3,
             .param .u32 .ptr.align 16 param4 // generic address
                                                // pointer
) { .. }
```

- `.param` → declares a kernel parameter.
- `.type` → type of the parameter (e.g., `.u32`, `.f32`, `.u64`).
- `.ptr` → indicates that the parameter is a **pointer**.
- `.space` → optional; indicates the **state space** the pointer points to: `.const`, `.global`, `.local`, `.shared`.
- `.align N` → optional; specifies **alignment** of the memory being pointed to (must be a power of two).

- `varname` → the name of the parameter.
- `.align N` guarantees the pointer points to memory aligned to **N bytes**.
- If omitted, alignment defaults to **4 bytes**.

PTX cannot pass large C structures in registers directly if they exceed 8 bytes. Instead, you **flatten the structure into a byte array** in `.param`.

```
struct MyStruct {
    double d;
    int y;
};
// Device function receiving a flattened struct
.func foo(
    .reg .b32 N,                // example scalar parameter
    .param .align 8 .b8 buffer[12] // 8 bytes double + 4 bytes int
)
{
    .reg .f64 %d;
    .reg .s32 %y;

    // Load members from parameter byte array
    ld.param.f64 %d, [buffer];    // double occupies first 8 bytes
    ld.param.s32 %y, [buffer+8];  // int occupies next 4 bytes
}

caller side:
.reg .f64 dbl;
.reg .s32 x;
.param .align 8 .b8 mystruct[12];

// Store values into flattened struct
st.param.f64 [mystruct+0], dbl; // double
st.param.s32 [mystruct+8], x;   // int

// Call function with struct
call foo, (4, mystruct);
```



```

.func foo(
    .param .b32 N,          // number of elements
    .param .b32 buffer[32]  // array of up to 8 floats (4 bytes each)
)
{
    .reg .u32  %n, %r;
    .reg .f32  %f;
    .reg .pred %p;

    ld.param.u32 %n, [N];      // load array length
    mov.u32      %r, buffer;   // get address of buffer in .local memory

Loop:
    setp.eq.u32  %p, %n, 0;
    @%p bra Done;              // exit when n==0

    ld.local.f32 %f, [%r];     // load float from local copy
    ...
    add.u32 %r, %r, 4;         // move to next float
    sub.u32 %n, %n, 1;
    bra Loop;

Done:
    ...
}

```

- **Arrays and structs larger than 8 bytes:** must use `.param` arrays.
- **Address-taking:** forces a **copy to `.local` memory**, then you access with `ld.local` / `st.local`.

Explicit declarations of variables in the texture state space is deprecated, and programs should instead reference texture memory through variables of type `.texref`. The `.tex` directive is retained for backward compatibility, and variables declared in the `.tex` state space are equivalent to module-scoped `.texref` variables in the `.global` state space.

```
.global .texref tex_a;
```

5. Types:

Basic Type	Fundamental Type Specifiers
Signed integer	<code>.s8</code> , <code>.s16</code> , <code>.s32</code> , <code>.s64</code>

Basic Type	Fundamental Type Specifiers
Unsigned integer	<code>.u8</code> , <code>.u16</code> , <code>.u32</code> , <code>.u64</code>
Floating-point	<code>.f16</code> , <code>.f16x2</code> , <code>.f32</code> , <code>.f64</code>
Bits (untyped)	<code>.b8</code> , <code>.b16</code> , <code>.b32</code> , <code>.b64</code> , <code>.b128</code>
Predicate	<code>.pred</code>

more detailed:

Category	PTX Specifier	Size	Usage / Restrictions	Notes / Example
Signed integers	<code>.s8</code> , <code>.s16</code> , <code>.s32</code> , <code>.s64</code>	8,16,32,64 bits	Compatible with same size unsigned; <code>.s8</code> and <code>.s16</code> restricted to <code>ld</code> , <code>st</code> , <code>cvt</code>	<code>mov.s32 %r1, %r2</code>
Unsigned integers	<code>.u8</code> , <code>.u16</code> , <code>.u32</code> , <code>.u64</code>	8,16,32,64 bits	Same as signed; <code>.u8</code> and <code>.u16</code> restricted	<code>add.u16 %r1, %r2, %r3</code>
Floating-point	<code>.f16</code> , <code>.f16x2</code> , <code>.f32</code> , <code>.f64</code>	16, 32, 32/16 packed, 32, 64 bits	<code>.f16</code> allowed only in conversions, half-precision instructions, texture fetch; <code>.f16x2</code> for half-precision arithmetic	<code>cvt.f32.f16 %f1, %f2</code>
Bits / untyped	<code>.b8</code> , <code>.b16</code> , <code>.b32</code> , <code>.b64</code> , <code>.b128</code>	8,16,32,64,128 bits	Can hold any type of same bit size	<code>ld.b32 %r1, [%ptr]</code>
Predicate	<code>.pred</code>	1 bit	Used in guard predicates	<code>@%p1 add.f32 %f1, %f2, %f3</code>
Alternate FP: bf16	<code>.b16</code>	16 bits	8-bit exponent, 7-bit mantissa; use <code>.b16</code> register	<code>ld.global.b16 %r1, [%ptr]</code>
Alternate FP: e4m3	<code>.b8</code>	8 bits	4-bit exponent, 3-bit mantissa; no infinity, limited NaN	Packed values for arrays
Alternate FP: e5m2	<code>.b8</code>	8 bits	5-bit exponent, 2-bit mantissa; no infinity/NaN	Use <code>.b8</code> registers
Alternate FP: tf32	<code>.b32</code>	32 bits	Matrix multiply & accumulate instructions; reduced precision (~10 bits)	Conversion from <code>.f32</code> allowed
Alternate FP: e2m1	<code>.b8</code>	4 bits per value, packed as <code>e2m1x2</code>	2-bit exponent, 1-bit mantissa; no infinity/NaN	2 values per <code>.b8</code>
Alternate FP: e2m3	<code>.b16</code>	6 bits per value, packed as <code>e2m3x2</code>	2-bit exponent, 3-bit mantissa; no infinity/NaN; MSB padded	2 values per <code>.b16</code>
Alternate FP: e3m2	<code>.b16</code>	6 bits per value, packed	3-bit exponent, 2-bit mantissa; no infinity/NaN; MSB padded	2 values per <code>.b16</code>

Category	PTX Specifier	Size	Usage / Restrictions	Notes / Example
		as <code>e3m2x2</code>		
Alternate FP: ue8m0	<code>.b16</code>	8 bits per value, packed as <code>ue8m0x2</code>	8-bit exponent, 0 mantissa; no infinity	2 values per <code>.b16</code>
Alternate FP: ue4m3	<code>.b8</code>	7 bits	4-bit exponent, 3-bit mantissa; no infinity; MSB padded	Single value per <code>.b8</code>
Fixed-point: s2f6	<code>.s8</code>	8 bits	2 integer bits, 6 fractional bits; signed 2's complement	Value = <code>s8 * 2⁻⁶</code> ; max 1.984375, min -2.0

6. Variables:

In PTX, a variable declaration describes both the variable's type and its state space. In addition to fundamental types, PTX supports types for simple aggregate objects such as vectors and arrays. Variables in PTX must declare **state space + type + name**. Optional array size, initializer, or alignment may appear.

```
.global .u32 loc;           // 32-bit unsigned integer in global memory
.reg    .s32 i;            // 32-bit signed integer in register
.const  .f32 bias[] = {-1.0, 1.0}; // array of floats in constant memory
.global .u8  bg[4] = {0,0,0,0}; // 4-byte array initialized to zeros
.reg    .v4  .f32 accel;    // vector of 4 floats in registers
.reg    .pred p, q, r;      // predicates (1-bit) in registers
```

- Prefix `.v2` or `.v4` for vectors of length 2 or 4.
- Must be a non-predicate type.
- Cannot exceed 128 bits.

```
.global .v4 .f32 V;        // 4-element float vector in global memory
.shared .v2 .u16 uv;       // 2-element 16-bit vector in shared memory
.global .v4 .b8 v;         // 4-byte vector in global memory
```

```
.local  .u16 kernel[19][19]; // 361 elements, 722 bytes total
.shared .u8  mailbox[128];   // 128-byte array in shared memory
// first dimension can be deduced from initializer
.global .u32 index[] = {0,1,2,3,4,5,6,7}; // 8 elements
.global .s32 offset[][2] = {{-1,0},{0,-1},{1,0},{0,1}}; // 4x2 array
.const .f32 vals[8] = {0.33, 0.25, 0.125};
// equivalent to {0.33, 0.25, 0.125, 0,0,0,0,0}
.global .s32 x[3][2] = {{1,2},{3}};
```

```
// equivalent to {{1,2},{3,0},{0,0}}
```

```
//variables can be initialized with addresses:
.global .u32 foo = 42;
.global .u32 p1 = foo;           // offset in .const space
.global .u32 p2 = generic(foo); // generic address of foo
.global .u32 parr[] = { generic(bar), generic(bar)+4, generic(bar)+8 };
mask() operator (PTX 7.1+) can extract bytes from addresses or constants f
or initialization:
.global .u8 addr[] = {0xFF(foo), 0xFF00(foo+4)};

//Shorthand for declaring multiple variables with a numeric suffix:
.reg .b32 %r<100>; // declares %r0, %r1, ..., %r99
Works with any fundamental type and state space.
Cannot declare arrays this way, no initializers allowed.
```

7. Operands:

```
.shared .u16 x;
.reg    .u16 r0;
.global .v4 .f32 V;
.reg    .v4 .f32 W;
.const  .s32 tbl[256];
.reg    .b32 p;
.reg    .s32 q;

ld.shared.u16    r0,[x];
ld.global.v4.f32 W, [V];
ld.const.s32     q, [tbl+12];
mov.u32         p, tbl;
```

8. Instruction Set

PTX instructions generally have from zero to four operands, plus an optional guard predicate appearing after an `@` symbol to the left of the `opcode`:

- `@p opcode;`
- `@p opcode a;`
- `@p opcode d, a;`
- `@p opcode d, a, b;`
- `@p opcode d, a, b, c;`

The `setp` instruction writes two destination registers. We use a `|` symbol to separate multiple destination registers.

```
setp.lt.s32 p|q, a, b; // p = (a < b); q = !(a < b);
```

For some instructions the destination operand is optional. A *bit bucket* operand denoted with an underscore (`_`) may be used in place of a destination register.

In PTX, predicate registers are virtual and have `.pred` as the type specifier. So, predicate registers can be declared as

```
.reg .pred p, q, r;
```

Meaning	Signed Operator	Unsigned Operator	Bit-Size Operator
<code>a == b</code>	<code>eq</code>	<code>eq</code>	<code>eq</code>
<code>a != b</code>	<code>ne</code>	<code>ne</code>	<code>ne</code>
<code>a < b</code>	<code>lt</code>	<code>lo</code>	n/a
<code>a <= b</code>	<code>le</code>	<code>ls</code>	n/a
<code>a > b</code>	<code>gt</code>	<code>hi</code>	n/a
<code>a >= b</code>	<code>ge</code>	<code>hs</code>	n/a

Operator	Meaning
<code>.ca</code>	Cache at all levels, likely to be accessed again. The default load instruction cache operation is <code>ld.ca</code> , which allocates cache lines in all levels (L1 and L2) with normal eviction policy. Global data is coherent at the L2 level, but multiple L1 caches are not coherent for global data. If one thread stores to global memory via one L1 cache, and a second thread loads that address via a second L1 cache with <code>ld.ca</code> , the second thread may get stale L1 cache data, rather than the data stored by the first thread. The driver must invalidate global L1 cache lines between dependent grids of parallel threads. Stores by the first grid program are then correctly fetched by the second grid program issuing default <code>ld.ca</code> loads cached in L1.
<code>.cg</code>	Cache at global level (cache in L2 and below, not L1). Use <code>ld.cg</code> to cache loads only globally, bypassing the L1 cache, and cache only in the L2 cache.
<code>.cs</code>	Cache streaming, likely to be accessed once. The <code>ld.cs</code> load cached streaming operation allocates global lines with evict-first policy in L1 and L2 to limit cache pollution by temporary streaming data that may be accessed once or twice. When <code>ld.cs</code> is applied to a Local window address, it performs the <code>ld.lu</code> operation.
<code>.lu</code>	Last use. The compiler/programmer may use <code>ld.lu</code> when restoring spilled registers and popping function stack frames to avoid needless write-backs of lines that will not be used again. The <code>ld.lu</code> instruction performs a load cached streaming operation (<code>ld.cs</code>) on global addresses.
<code>.cv</code>	Don't cache and fetch again (consider cached system memory lines stale, fetch again). The <code>ld.cv</code> load operation applied to a global System Memory address invalidates (discards) a matching L2 line and re-fetches the line on each new load.

Operator	Meaning
<code>.wb</code>	Cache write-back all coherent levels. The default store instruction cache operation is <code>st.wb</code>, which writes back cache lines of coherent cache levels with normal eviction policy. If one thread stores to global memory, bypassing its L1 cache, and a second thread in a different SM later loads from that address via a different L1 cache with <code>ld.ca</code>, the second thread may get a hit on stale L1 cache data, rather than get the data from L2 or memory stored by the first thread. The driver must invalidate global L1 cache lines between dependent grids of thread arrays. Stores by the first grid program are then correctly missed in L1 and fetched by the second grid program issuing default <code>ld.ca</code> loads.
<code>.cg</code>	Cache at global level (cache in L2 and below, not L1). Use <code>st.cg</code> to cache global store data only globally, bypassing the L1 cache, and cache only in the L2 cache.
<code>.cs</code>	Cache streaming, likely to be accessed once. The <code>st.cs</code> store cached-streaming operation allocates cache lines with evict-first policy to limit cache pollution by streaming output data.
<code>.wt</code>	Cache write-through (to system memory). The <code>st.wt</code> store write-through operation applied to a global System Memory address writes through the L2 cache.

Notes for later:

so in the data preprocessing phase:

i need to remove any line that start with `//` or `/* */` because its certainly a comment

and i need to remove the first directives because they are constant across the whole dataset

`.version` `.target` `.address_size`

-i need to search in all the ptx files that i have for `.tex` and `.atom` and tensors:

`wmma.mma`, `mma` and `wgmma.mma_async`

-are shared and registers numbers present in the PTX the same as retrieved by `—ptxas infos?` if yes then we should reduce dimensions right? i will ensure it later